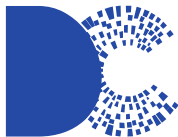




UNIVERSIDADE
FEDERAL DE
SERGIPE



DEPARTAMENTO
DE COMPUTAÇÃO

Gerenciadores de dispositivos

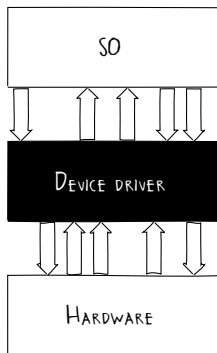
Interface Hardware/Software

Bruno Prado

Departamento de Computação / UFS

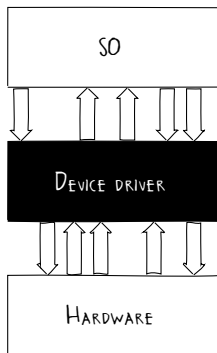
Introdução

- ▶ O que é um gerenciador de dispositivo?
 - ▶ É o código fonte utilizado para um hardware específico (*device driver*) através de uma API
 - ▶ Abstrai do resto do sistema os detalhes de como o dispositivo funciona (caixa preta)



Introdução

- ▶ O que é um gerenciador de dispositivo?
 - ▶ É o código fonte utilizado para um hardware específico (*device driver*) através de uma API
 - ▶ Abstrai do resto do sistema os detalhes de como o dispositivo funciona (caixa preta)



Modularidade $\longleftrightarrow$ *Plug and play*

Introdução

- ▶ Quais são os tipos de gerenciadores de dispositivo?

Introdução

- ▶ Quais são os tipos de gerenciadores de dispositivo?
 - ▶ *Caractere*: neste tipo de dispositivo o acesso é feito por fluxo de bytes, assim como ler ou escrever em um arquivo, utilizando as chamadas de sistema
 - ▶ Terminal de texto (**/dev/console**)
 - ▶ Gerador de números aleatórios (**/dev/random**)

Introdução

- ▶ Quais são os tipos de gerenciadores de dispositivo?
 - ▶ *Caractere*: neste tipo de dispositivo o acesso é feito por fluxo de bytes, assim como ler ou escrever em um arquivo, utilizando as chamadas de sistema
 - ▶ Terminal de texto (**/dev/console**)
 - ▶ Gerador de números aleatórios (**/dev/random**)
 - ▶ *Bloco*: permite que sistemas de arquivo sejam gerenciados através de blocos com tamanho múltiplo de 2 e com acesso transparente ao usuário
 - ▶ Cartão SD (**/dev/mmcblk0**)
 - ▶ Disco SATA (**/dev/sda**)

Introdução

- ▶ Quais são os tipos de gerenciadores de dispositivo?
 - ▶ *Caractere*: neste tipo de dispositivo o acesso é feito por fluxo de bytes, assim como ler ou escrever em um arquivo, utilizando as chamadas de sistema
 - ▶ Terminal de texto (**/dev/console**)
 - ▶ Gerador de números aleatórios (**/dev/random**)
 - ▶ *Bloco*: permite que sistemas de arquivo sejam gerenciados através de blocos com tamanho múltiplo de 2 e com acesso transparente ao usuário
 - ▶ Cartão SD (**/dev/mmcblk0**)
 - ▶ Disco SATA (**/dev/sda**)
 - ▶ *Rede*: proporciona a troca de dados com outros sistemas através da transmissão e no recebimento de pacotes de dados, com interface distinta da utilizada pelos dispositivos de caractere ou de bloco (*socket*)
 - ▶ Interface de *loopback*
 - ▶ Dispositivo *ethernet*

Introdução

- ▶ Por que desenvolver um gerenciador de dispositivo?

Introdução

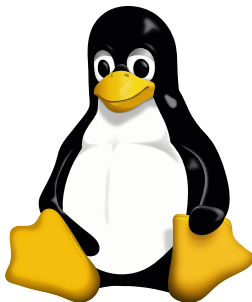
- ▶ Por que desenvolver um gerenciador de dispositivo?
 - ▶ Suportar para dispositivos personalizados
 - ▶ Especificações de hardware em plataformas virtuais
 - ▶ Avaliação de protótipos do sistema
 - ▶ Emulação de periféricos

Introdução

- ▶ Por que desenvolver um gerenciador de dispositivo?
 - ▶ Suportar para dispositivos personalizados
 - ▶ Especificações de hardware em plataformas virtuais
 - ▶ Avaliação de protótipos do sistema
 - ▶ Emulação de periféricos
 - ▶ Permitir o uso de equipamentos não suportados
 - ▶ Fabricante não disponibiliza o *device driver* para o SO
 - ▶ Limitações ou não adequações de funcionalidades
 - ▶ Hardware legado incompatível com novas versões

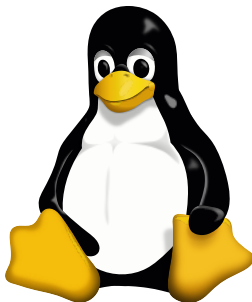
Introdução

- ▶ Por que usar Linux?



Introdução

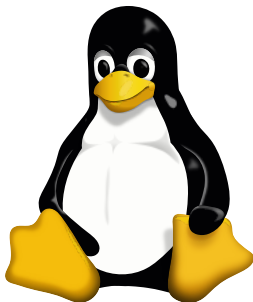
- ▶ Por que usar Linux?



- ▶ Código fonte aberto e livre (GPLv2)

Introdução

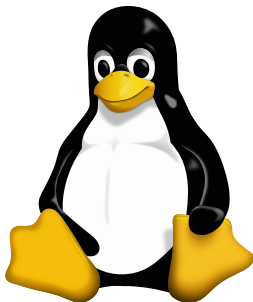
- ▶ Por que usar Linux?



- ▶ Código fonte aberto e livre (GPLv2)
- ▶ Aplicações complexas, modulares e portáteis

Introdução

- ▶ Por que usar Linux?



- ▶ Código fonte aberto e livre (GPLv2)
- ▶ Aplicações complexas, modulares e portáteis
- ▶ Utilizado em 80% dos servidores e presente em mais da metade dos dispositivos embarcados já produzidos

Introdução

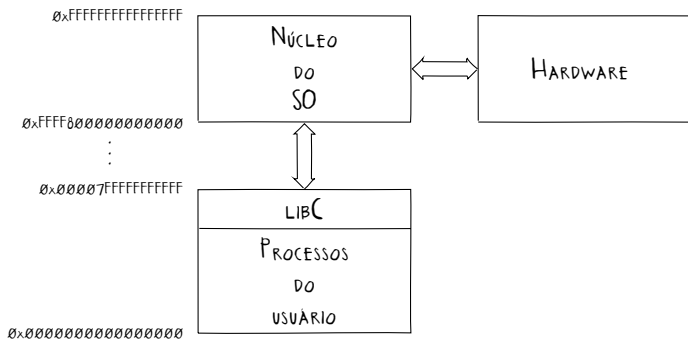
► Principais diretórios do núcleo do Linux

/	
├── arch	Código fonte específicos de arquitetura (arm, ia64, mips, riscv, x86, xtensa, ...)
├── block	Dispositivos de armazenamento por bloco
├── Documentation	Descrição das APIs do núcleo do Linux
├── drivers	Gerenciadores de dispositivo (dma, usb, ...)
├── fs	Sistemas de arquivo suportados (EXT2/3/4, FAT, HFS/HPS+, NFS, NTFS, RAMFS, UFS, XFS, ...)
├── include	Cabeçalhos do núcleo do Linux
├── init	Rotinas de inicialização e de <i>start up</i>
├── kernel	Código fonte independente de arquitetura
├── mm	Gerenciamento de memória
└── ...	

<https://github.com/torvalds/linux>

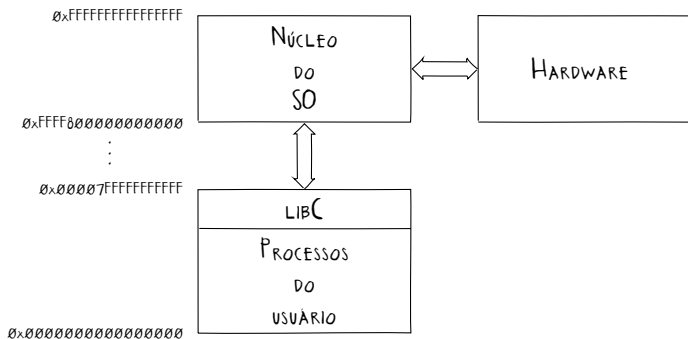
Introdução

- ▶ Espaço do usuário × Espaço do núcleo
 - ▶ O núcleo do SO roda em espaço de memória protegido e em modo privilegiado
 - ▶ Cada processo do usuário executa encapsulados pela memória virtual e com recursos dedicados



Introdução

- ▶ Espaço do usuário × Espaço do núcleo
 - ▶ O núcleo do SO roda em espaço de memória protegido e em modo privilegiado
 - ▶ Cada processo do usuário executa encapsulados pela memória virtual e com recursos dedicados



A troca de dados entre os espaços de memória é feita através de chamadas de sistema (*system calls*) que funcionam utilizando interrupções de software

Introdução

- ▶ Módulos do Linux
 - ▶ São componentes que rodam no espaço do núcleo para estender as funcionalidades do sistema dinamicamente (*CONFIG_MODULES*)
 - ▶ **insmod /caminho/do/driver.ko**: utilizado para carregar módulo durante o seu desenvolvimento
 - ▶ **modprobe driver**: carrega as dependências necessárias para o módulo antes ser carregado
 - ▶ **rmmod driver**: remove o módulo do sistema (*CONFIG_MODULE_UNLOAD*) e com a opção **-f** força a remoção (*MODULE_FORCE_UNLOAD*), desde que as suas respectivas definições estejam habilitadas
 - ▶ **modprobe -r driver**: descarrega o módulo, verificando e removendo as dependências não utilizadas
 - ▶ **lsmod**: lista os módulos carregados no sistema

Estrutura do módulo

▶ hello_world.c

```
1 // Cabeçalhos do Linux
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4 #include <linux/module.h>
5 MODULE_AUTHOR("Bruno Prado");
6 MODULE_DESCRIPTION("Hello_world_module_example");
7 MODULE_INFO(difficulty, "VERY_EASY");
8 MODULE_LICENSE("GPL");
9 static __init int hello_world_init(void) {
10     pr_info("Hello_world!\n");
11     return 0;
12 }
13 static __exit void hello_world_exit(void) {
14     pr_info("Goodbye_world!\n");
15 }
16 module_init(hello_world_init);
17 module_exit(hello_world_exit);
```

<https://github.com/torvalds/linux/tree/master/include/linux>

Estrutura do módulo

► hello_world.c

```
1 // Cabeçalhos do Linux
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4 #include <linux/module.h>
5 MODULE_AUTHOR("Bruno Prado");
6 MODULE_DESCRIPTION("Hello_world_module_example");
7 MODULE_INFO(difficulty, "VERY_EASY");
8 MODULE_LICENSE("GPL");
9 static __init int hello_world_init(void) {
10     pr_info("Hello_world!\n");
11     return 0;
12 }
13 static __exit void hello_world_exit(void) {
14     pr_info("Goodbye_world!\n");
15 }
16 module_init(hello_world_init);
17 module_exit(hello_world_exit);
```

#define __init__section(.init.text)

#define __exit__section(.exit.text)

Estrutura do módulo

► hello_world.c

```
1 // Cabeçalhos do Linux
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4 #include <linux/module.h>
5 MODULE_AUTHOR("Bruno Prado");
6 MODULE_DESCRIPTION("Hello_world_module_example");
7 MODULE_INFO(difficulty, "VERY_EASY");
8 MODULE_LICENSE("GPL");
9 static __init int hello_world_init(void) {
10     pr_info("Hello_world!\n");
11     return 0;
12 }
13 static __exit void hello_world_exit(void) {
14     pr_info("Goodbye_world!\n");
15 }
16 module_init(hello_world_init);
17 module_exit(hello_world_exit);
```

Informações do módulo

"GPL", "GPL v2", ..., "Proprietary"

Estrutura do módulo

▶ hello_world.c

```
1 // Cabeçalhos do Linux
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4 #include <linux/module.h>
5 MODULE_AUTHOR("Bruno Prado");
6 MODULE_DESCRIPTION("Hello_world_module_example");
7 MODULE_INFO(difficulty, "VERY_EASY");
8 MODULE_LICENSE("GPL");
9 static __init int hello_world_init(void) {
10     pr_info("Hello_world!\n");
11     return 0;
12 }
13 static __exit void hello_world_exit(void) {
14     pr_info("Goodbye_world!\n");
15 }
16 module_init(hello_world_init);
17 module_exit(hello_world_exit);
```

Mensagem informativa do núcleo (printk KERN_INFO)

Construção do módulo

► Makefile

```
1 obj-m := hello_world.o
2 KDIR   ?= /lib/modules/$(shell uname -r)/build
3 all: modules
4 install: modules_install
5 modules modules_install help clean:
6     $(MAKE) -C $(KDIR) M=$(shell pwd) $(@)
```

<https://github.com/torvalds/linux/.../Documentation/kbuild>

Construção do módulo

► Compilação com Makefile

```
$ make
make -C /lib/modules/5.1.9-arch1-1-ARCH/build M=/home/bruno/ihs modules
make[1]: Entering directory '/usr/lib/modules/5.1.9-arch1-1-ARCH/build'
  CC [M] /home/bruno/ihs/hello_world.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC /home/bruno/ihs/hello_world.mod.o
  LD [M] /home/bruno/ihs/hello_world.ko
make[1]: Leaving directory '/usr/lib/modules/5.1.9-arch1-1-ARCH/build'
$
```


Construção do módulo

► Compilação com Makefile

```
$ make
make -C /lib/modules/5.1.9-arch1-1-ARCH/build M=/home/bruno/ihs modules
make[1]: Entering directory '/usr/lib/modules/5.1.9-arch1-1-ARCH/build'
  CC [M] /home/bruno/ihs/hello_world.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC /home/bruno/ihs/hello_world.mod.o
  LD [M] /home/bruno/ihs/hello_world.ko
make[1]: Leaving directory '/usr/lib/modules/5.1.9-arch1-1-ARCH/build'
$ modinfo hello_world.ko
filename:      /home/bruno/ihs/hello_world.ko
license:      GPL
difficulty:   VERY EASY
description:   Hello world module example
author:       Bruno Prado
srcversion:   965458B4C080D949518B587
depends:
retpoline:    Y
name:         hello_world
vermagic:     5.1.9-arch1-1-ARCH SMP preempt mod_unload
$
```

Seção do módulo

► Exibição de seção do informações do módulo

```
$ objdump -d --section=,modinfo hello_world.ko

hello_world.ko: formato de arquivo elf64-x86-64

Desmontagem da seção .modinfo:

0000000000000000 <__UNIQUE_ID_license21>:
   0: 6c 69 63 65 6e 73 65 3d 47 50 4c 00          license=GPL.

000000000000000c <__UNIQUE_ID_difficulty20>:
   c: 64 69 66 66 69 63 75 6c 74 79 3d 56 45 52 59 20 difficulty=VERY
  1c: 45 41 53 59 00                               EASY.

0000000000000021 <__UNIQUE_ID_author19>:
  21: 61 75 74 68 6f 72 3d 42 72 75 6e 6f 20 50 72 61   author=Bruno Pra
  31: 64 6f 00 00 00 00 00                               do....

0000000000000038 <__UNIQUE_ID_srcversion22>:
  38: 73 72 63 76 65 72 73 69 6f 6e 3d 39 36 35 34 35   srcversion=96545
  48: 38 42 34 43 30 38 30 44 39 34 39 35 31 38 42 35   8B4C080D949518B5
  58: 38 37 00 00 00 00 00                               87.....
```

Seção do módulo

► Exibição de seção do informações do módulo

```
0000000000000060 <__module_depends>:
 60: 64 65 70 65 6e 64 73 3d 00          depends=.

0000000000000069 <__UNIQUE_ID_retpoline21>:
 69: 72 65 74 70 6f 6c 69 6e 65 3d 59 00      retpoline=Y.

0000000000000075 <__UNIQUE_ID_name20>:
 75: 6e 61 6d 65 3d 68 65 6c 6c 6f 5f 77 6f 72 6c 64      name=hello_world ...

0000000000000086 <__UNIQUE_ID_vermagic19>:
 86: 76 65 72 6d 61 67 69 63 3d 35 2e 31 2e 39 2d 61      vermagic=5.1.9-a
 96: 72 63 68 31 2d 31 2d 41 52 43 48 20 53 4d 50 20      rch1-1-ARCH SMP
 a6: 70 72 65 65 6d 70 74 20 6d 6f 64 5f 75 6e 6c 6f      preempt mod_unlo
 b6: 61 64 20 00          ad .

$
```

Execução do módulo

► Inicialização e finalização

```
$ sudo -s  
...  
#
```

Execução do módulo

► Inicialização e finalização

```
$ sudo -s  
...  
# insmod hello_world.ko  
#
```

Execução do módulo

► Inicialização e finalização

```
$ sudo -s  
...  
# insmod hello_world.ko  
# rmmod hello_world  
#
```

Execução do módulo

► Inicialização e finalização

```
$ sudo -s
...
# insmod hello_world.ko
# rmmod hello_world
# dmesg | tail -n 2
[ 274.949203] Hello world!
[ 281.670835] Goodbye world!
#
```

Execução do módulo

► Inicialização e finalização

```
$ sudo -s
...
# insmod hello_world.ko
# rmmod hello_world
# dmesg | tail -n 2
[ 274.949203] Hello world!
[ 281.670835] Goodbye world!
# exit
$
```


Parâmetros do módulo

- ▶ A utilização de parâmetros permite ao módulo receber argumentos por linha comando
 - ▶ **module_param**(name, type, perm)
 - ▶ *name*: nome da variável
 - ▶ *type*: tipo da variável (**bool**, **charp**, **byte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**)
 - ▶ *perm*: permissões de acesso, com prefixo **S_Ixxxx** para leitura (**R**), escrita (**W**), execução (**X**) para usuário (**USR**), grupo (**GRP**) e usuário/grupo/outros (**UGO**)
 - ▶ **MODULE_PARM_DESC**(_parm, desc)
 - ▶ *_parm*: parâmetro do módulo
 - ▶ *desc*: descrição do parâmetro

Parâmetros do módulo

► hello_world_param.c

```
1 // Cabeçalhos do Linux
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4 #include <linux/module.h>
5 #include <linux/moduleparam.h>
6 ...
7
8
9 static int array[] = { 1, 2, 3 };
10 static char* string = "easy";
11 static int value = 123;
12 module_param_array(array, int, NULL, S_IRUGO);
13 module_param(string, charp, S_IRUGO);
14 module_param(value, int, S_IRUSR | S_IWUSR);
15 MODULE_PARM_DESC(array, "Array_of_integer");
16 MODULE_PARM_DESC(string, "String_message");
17 MODULE_PARM_DESC(value, "Integer_value");
18 ...
```

Parâmetros do módulo

► hello_world_param.c

```
1 // Cabeçalhos do Linux
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4 #include <linux/module.h>
5 #include <linux/moduleparam.h>
6 ...
7
18 static __init int hello_world_init(void) {
19     pr_info("Hello_world_with_parameters!\n");
20     pr_info("array={%i,%i,%i}\n", array[0],
21         array[1], array[2]);
22     pr_info("string=%s\n", string);
23     pr_info("value=%i\n", value);
24     return 0;
25 }
26
27 static __exit void hello_world_exit(void) {
28     pr_info("Goodbye_world_with_parameters!\n");
29 }
30
31 module_init(hello_world_init);
32 module_exit(hello_world_exit);
```

Construção do módulo

► Compilação e informações

```
$ make
make -C /lib/modules/5.1.9-arch1-1-ARCH/build M=/home/bruno/ihs modules
make[1]: Entering directory '/usr/lib/modules/5.1.9-arch1-1-ARCH/build'
  CC [M] /home/bruno/ihs/hello_world.o
  CC [M] /home/bruno/ihs/hello_world_param.o
Building modules, stage 2.
MODPOST 2 modules
  CC /home/bruno/ihs/hello_world.mod.o
  LD [M] /home/bruno/ihs/hello_world.ko
  CC /home/bruno/ihs/hello_world_param.mod.o
  LD [M] /home/bruno/ihs/hello_world_param.ko
make[1]: Leaving directory '/usr/lib/modules/5.1.9-arch1-1-ARCH/build'
$
```

Construção do módulo

► Compilação e informações

```
$ modinfo hello_world_param.ko
filename:      /home/bruno/ihs/hello_world_param.ko
license:      GPL
difficulty:    VERY EASY
description:   Hello world module with parameters example
author:       Bruno Prado
srcversion:    061EB00D5F2D9E0484DD0F7
depends:
retpoline:    Y
name:         hello_world_param
vermagic:     5.1.9-arch1-1-ARCH SMP preempt mod_unload
parm:         array:Integer array (array of int)
parm:         string:String message (charp)
parm:         value:Integer value (int)
$
```

Execução do módulo

- ▶ Inicialização e finalização com parâmetros

```
$ sudo -s  
...  
#
```

Execução do módulo

- ▶ Inicialização e finalização com parâmetros

```
$ sudo -s  
...  
# insmod hello_world_param.ko  
#
```

Execução do módulo

- ▶ Inicialização e finalização com parâmetros

```
$ sudo -s
...
# insmod hello_world_param.ko
# rmmod hello_world_param
#
```


Execução do módulo

► Inicialização e finalização com parâmetros

```
$ sudo -s
...
# insmod hello_world_param.ko
# rmmod hello_world_param
# dmesg | tail -n 5
[ 8029.792464] Hello world with parameters!
[ 8029.792467] array = 1, 2, 3
[ 8029.792468] string = easy
[ 8029.792468] value = 123
[ 8035.628040] Goodbye world with parameters!
#
```

Execução do módulo

► Inicialização e finalização com parâmetros

```
$ sudo -s
...
# insmod hello_world_param.ko
# rmmod hello_world_param
# dmesg | tail -n 5
[ 8029.792464] Hello world with parameters!
[ 8029.792467] array = 1, 2, 3
[ 8029.792468] string = easy
[ 8029.792468] value = 123
[ 8035.628040] Goodbye world with parameters!
# insmod hello_world_param.ko array=3,2,1 string="sooo easy" value=321
# rmmod hello_world_param
# dmesg | tail -n 5
[ 8404.266364] Hello world with parameters!
[ 8404.266368] array = 3, 2, 1
[ 8404.266370] string = sooo easy
[ 8404.266372] value = 321
[ 8407.131938] Goodbye world with parameters!
#
```

Execução do módulo

► Inicialização e finalização com parâmetros

```
$ sudo -s
...
# insmod hello_world_param.ko
# rmmod hello_world_param
# dmesg | tail -n 5
[ 8029.792464] Hello world with parameters!
[ 8029.792467] array = 1, 2, 3
[ 8029.792468] string = easy
[ 8029.792468] value = 123
[ 8035.628040] Goodbye world with parameters!
# insmod hello_world_param.ko array=3,2,1 string='"sooo easy"' value=321
# rmmod hello_world_param
# dmesg | tail -n 5
[ 8404.266364] Hello world with parameters!
[ 8404.266368] array = 3, 2, 1
[ 8404.266370] string = sooo easy
[ 8404.266372] value = 321
[ 8407.131938] Goodbye world with parameters!
# exit
$
```

Exercício

- ▶ Reproduza os experimentos realizados para construção e execução dos módulos do Linux
 - ▶ Verifique o tratamento dos erros e de ponteiros nulos
 - ▶ Estude a função *printk* e os seus níveis de prioridade
 - ▶ Explique os processos de compilação dos módulos no núcleo (*in-tree*) e externo (*out-of-tree*)